Resume Service Detailed Documentation

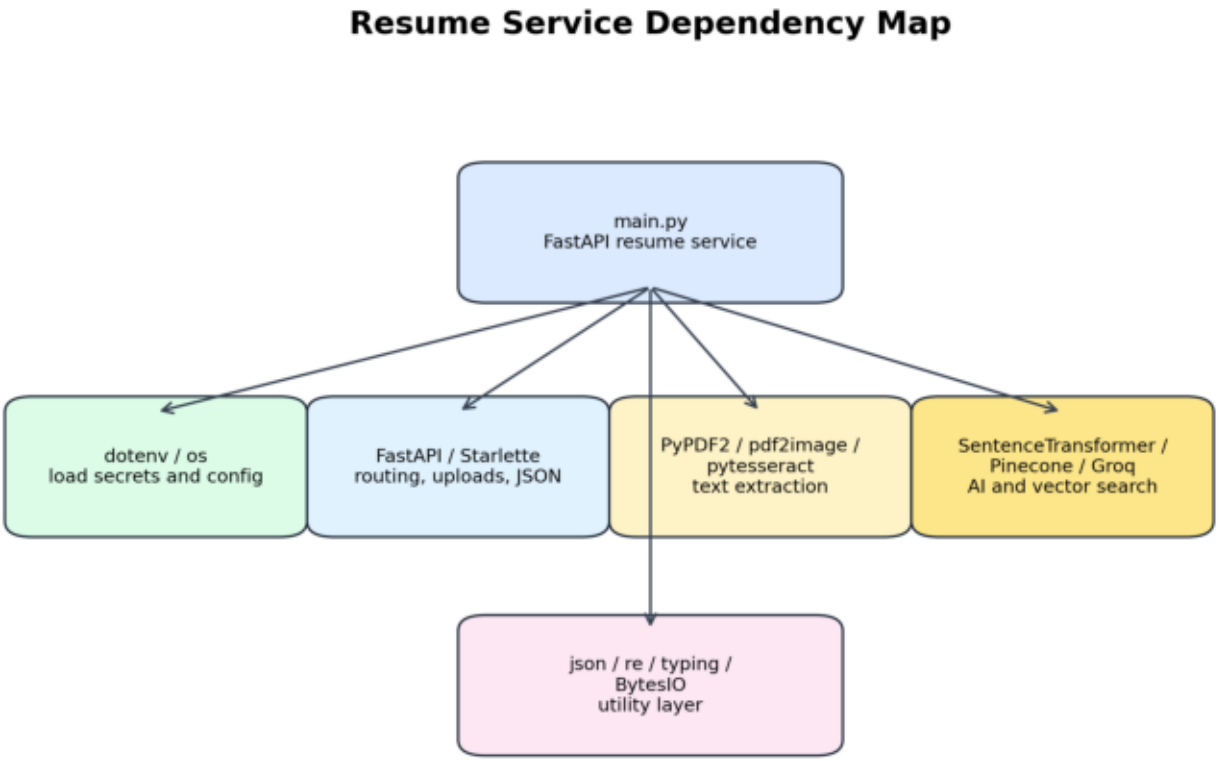
Deep dive into `resume/main.py`, its libraries, workflows, and design choices

- follows the Python file from top to bottom
- explains each imported library and why it is used
- documents every helper and endpoint
- includes diagrams and interview-ready talking points

Source scope

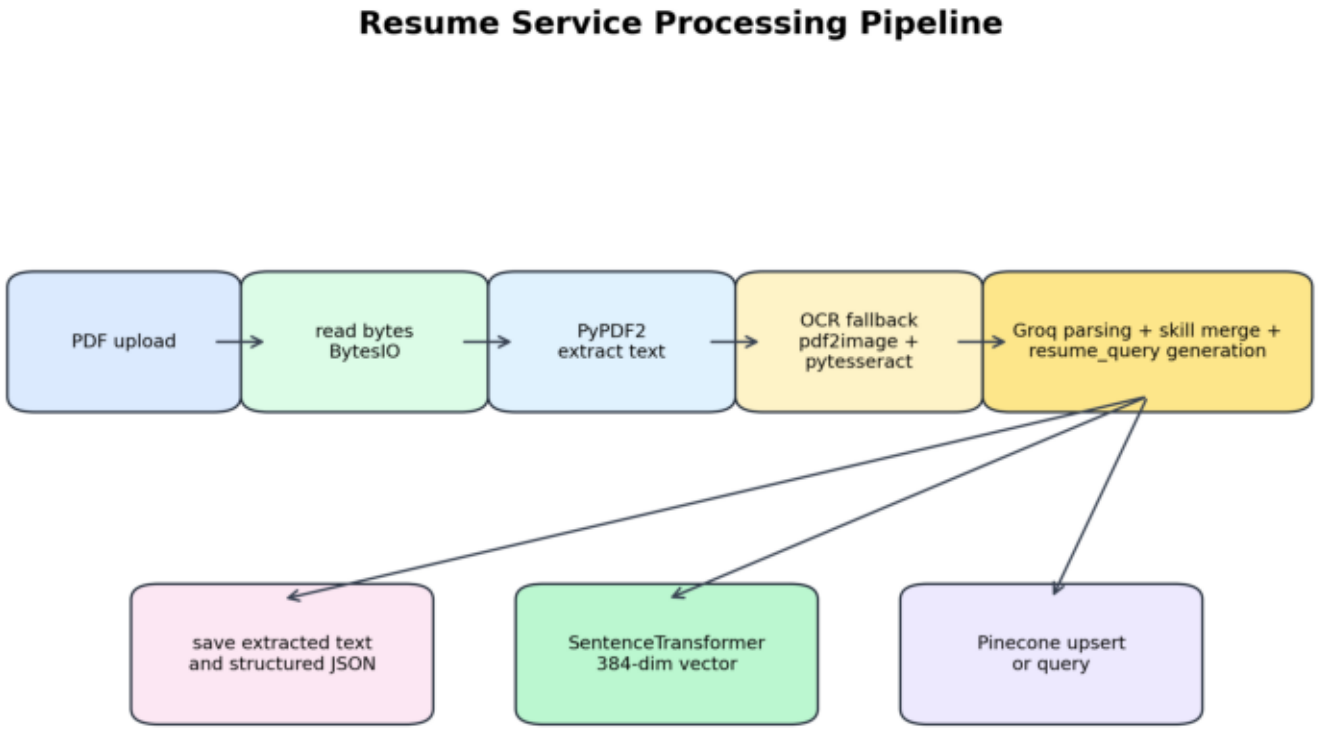
Primary code file: `resume/main.py`
Supporting context: local venv package set and generated outputs in `resume/extracted_texts`

Dependency Map



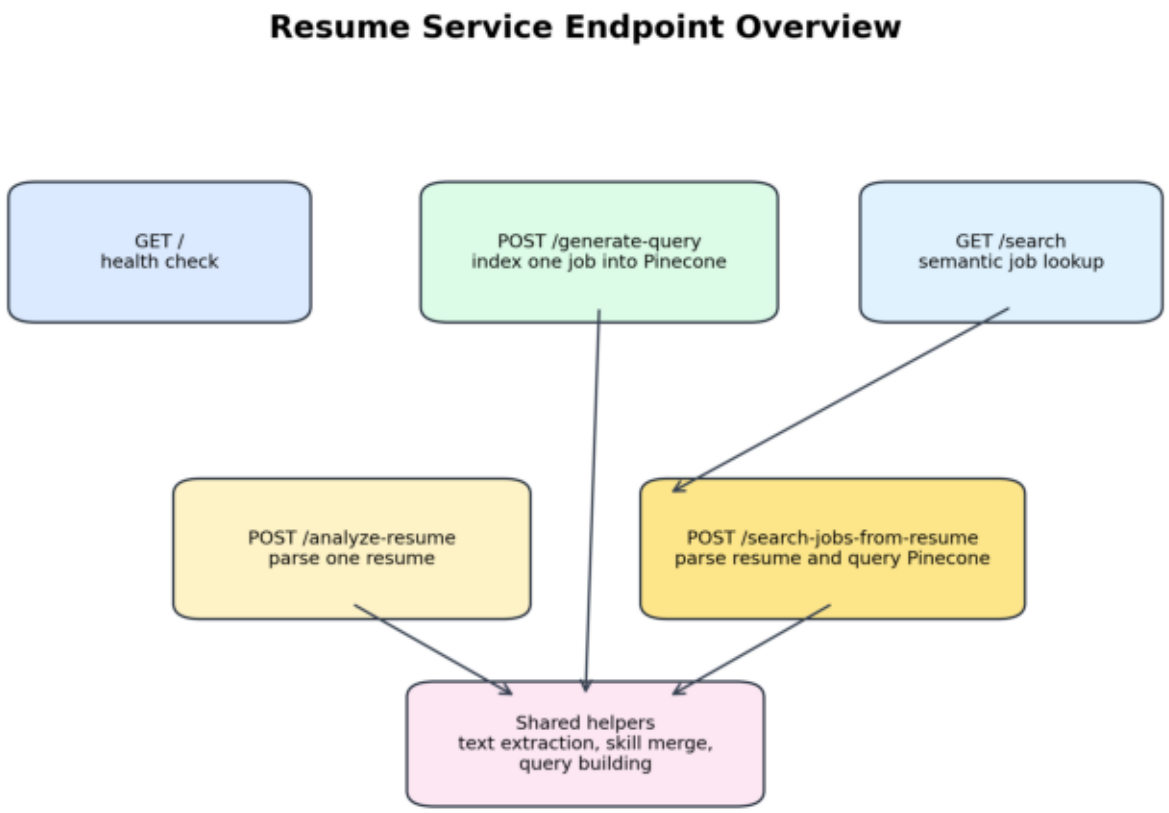
This map groups the imported libraries by responsibility: configuration, web routing, PDF/OCR extraction, semantic AI, and general utility logic.

Processing Pipeline



This is the main runtime flow for a resume: upload, extract text, OCR fallback, LLM structuring, semantic query generation, and optional vector storage or retrieval.

Endpoint Map



The service exposes one health endpoint and four functional endpoints. Most of the logic is shared through helper functions in the same file.

Service Overview

The `resume` folder contains a Python microservice whose job is to understand resumes and connect them to jobs semantically rather than by exact keyword match. The core file is `resume/main.py`, and nearly all of the service behavior lives in that one file.

What this service does:
- Accepts uploaded resume PDFs.
- Extracts readable text from normal PDFs.
- Falls back to OCR for scanned PDFs.
- Uses a large language model through Groq to turn raw text into structured resume JSON.
- Builds a semantic search query from that structured information.
- Generates embeddings using a sentence-transformer model.
- Stores job vectors in Pinecone.
- Searches Pinecone to find jobs similar to a candidate resume.

In the larger project, Spring Boot owns users and jobs in MySQL, while this Python service owns AI-heavy processing and vector search.

Folder Contents

The `resume` folder contains:
- `main.py`: the actual FastAPI application.
- `.env`: environment variables used for API keys and optional config. The code expects Pinecone and Groq keys here.
- `extracted_texts/`: generated output files created when resumes are analyzed. The service writes extracted text and structured JSON here.
- `venv/`: the local Python virtual environment used by the project.
- `temp.py`: a tiny scratch script unrelated to the production service.
- `__pycache__/`: compiled Python cache files.

Only `main.py` is part of the real service implementation. The rest are either runtime support, generated artifacts, or local development files.

Top Of File Imports And Why They Exist

The code begins with standard-library imports:
- `BytesIO` from `io`: lets the service treat in-memory file bytes like a file object. This is important because `PyPDF2.PdfReader` expects file-like input.
- `json`: used to parse the Groq model response and to save structured resume output as `.json`.
- `os`: used for reading environment variables, creating directories, and building file paths.
- `re`: used for text cleanup and for stripping markdown code fences from LLM output.
- `warnings`: used to suppress `FutureWarning`.
- `Any`, `Dict`, and `List` from `typing`: improve readability and communicate expected data shapes.

Third-party imports are grouped by function:
- `load_dotenv` from `dotenv`: loads secret keys from `.env` into environment variables.
- `FastAPI`, `File`, `HTTPException`, `UploadFile`: the main web framework and request helpers.
- `CORSMiddleware`: allows the frontend running on another port to call this backend in the browser.
- `JSONResponse`: returns explicit JSON payloads with status codes.
- `Groq`: Python SDK used to call the hosted LLM.
- `convert_from_bytes` from `pdf2image`: converts PDF pages to images so OCR can run on scanned documents.
- `Pinecone`: vector database client used for storing and searching embeddings.
- `BaseModel` from `pydantic`: defines request schemas with validation.
- `PdfReader` from `PyPDF2`: extracts text from text-based PDFs.
- `SentenceTransformer`: loads a pretrained embedding model for semantic search.
- `pytesseract`: wrapper around the Tesseract OCR engine.

Design takeaway:
Every imported library maps to a concrete stage in the pipeline: web serving, config loading, text extraction, OCR, LLM structuring, embedding generation, or vector search.

Library Versions In The Local Virtual Environment

Using the local virtual environment in `resume/venv`, the main installed package versions are:
- `fastapi 0.135.1`
- `starlette 0.52.1`
- `uvicorn 0.42.0`
- `pydantic 2.12.5`
- `python-dotenv 1.2.2`
- `PyPDF2 3.0.1`
- `pdf2image 1.17.0`
- `pytesseract 0.3.13`
- `sentence-transformers 5.3.0`
- `transformers 5.3.0`
- `torch 2.11.0`
- `groq 1.1.1`
- `pinecone 8.1.0`
- `pillow 12.1.1`

Why these matter:
- `FastAPI`, `Starlette`, and `Uvicorn` make the HTTP service run.
- `Pydantic` guarantees request payload shape.
- `PyPDF2`, `pdf2image`, `pytesseract`, and `Pillow` form the document ingestion stack.
- `sentence-transformers`, `transformers`, and `torch` form the ML stack for embeddings.
- `Groq` and `Pinecone` connect to external AI infrastructure.

Startup And Application Setup

Immediately after imports, the code does two startup-time things:
- `warnings.filterwarnings("ignore", category=FutureWarning)`
- `load_dotenv()`

The warning filter keeps startup logs quieter, especially for ML-related dependencies that may emit future-facing notices. This is mostly for developer convenience.

Then the app object is created:
- `app = FastAPI(title="Resume Analyzer + Job Query Generator")`

This line defines the ASGI application. FastAPI uses this object to register routes, generate OpenAPI docs, and serve requests.

Next, CORS middleware is added. This is necessary because the frontend runs on `localhost:5173`, which is a different origin from the Python service running on another port. Without CORS, browser requests from the frontend would be blocked.

The configured CORS policy:
- explicitly allows `http://localhost:5173`
- explicitly allows `http://127.0.0.1:5173`
- also allows matching localhost-style origins through a regex
- allows credentials
- allows all methods and all headers

This is a development-friendly policy. It is flexible, but in production you would normally narrow it to trusted origins only.

Tesseract Configuration And Output Directory

The line:
- `pytesseract.pytesseract.tesseract_cmd = r"C:\Program Files\Tesseract-OCR\tesseract.exe"`

exists because `pytesseract` is only a Python wrapper. The real OCR engine is the Tesseract executable installed on the machine. If that executable is not already in the system PATH, the code must tell `pytesseract` exactly where it lives.

Why this matters:
- Without Tesseract installed, scanned PDFs cannot be processed.
- Text-based PDFs may still work because `PyPDF2` does not need Tesseract.

The service also sets:
- `OUTPUT_DIR = "extracted_texts"`
- `os.makedirs(OUTPUT_DIR, exist_ok=True)`

This ensures there is always a local directory where extracted plain text and structured JSON results can be written. It makes debugging much easier because every analyzed resume leaves behind inspectable artifacts.

Environment Variables And External Clients

The service reads:
- `PINECONE_API_KEY`
- `PINECONE_INDEX_NAME`, defaulting to `quickstart`
- `PINECONE_NAMESPACE`, defaulting to `job-queries`
- `GROQ_API_KEY`

Then it immediately validates that the two required secrets exist:
- if `PINECONE_API_KEY` is missing, startup fails
- if `GROQ_API_KEY` is missing, startup fails

This is good because it fails fast. The service does not wait until the first user request to discover that a critical dependency cannot be reached.

After that, the external clients are initialized:
- `pc = Pinecone(api_key=PINECONE_API_KEY)`
- `index = pc.Index(PINECONE_INDEX_NAME)`
- `embedding_model = SentenceTransformer("all-MiniLM-L6-v2")`
- `groq_client = Groq(api_key=GROQ_API_KEY)`

What each object does:
- `pc` is the root Pinecone SDK client.
- `index` is the handle for one logical vector index.
- `embedding_model` is a local transformer model that converts text into dense numeric vectors.
- `groq_client` is the network client used to send prompts to Groq-hosted models.

Operational implication:
The model and the external clients are created once at startup and reused across requests, which is much more efficient than recreating them for every API call.

Pydantic Model: JobRequest

The first custom model is:
- `class JobRequest(BaseModel)`

It declares:
- `id: int`
- `title: str`
- `company: str`
- `location: str`
- `skills: List[str]`

Why this class exists:
- FastAPI uses Pydantic models to validate incoming JSON automatically.
- It ensures the `/generate-query` endpoint receives the shape the code expects.
- It also improves the generated API docs because FastAPI can describe the schema.

Conceptually, this model is the bridge between the Java backend and the Python AI layer. Spring Boot creates job records in MySQL, then sends this exact structure here so the Python service can turn the job into an embedding and store it in Pinecone.

Job Query Helper Functions

The first helper block is about jobs and Pinecone indexing.

`build_query(job: JobRequest) -> str`
- Joins the list of skills into a single string.
- Builds a sentence like `Software Engineer at ABC Corp in Pune with skills python react sql`.
- This sentence is important because embedding models work on natural-language-style text, not raw object fields.

`generate_embedding(text: str) -> List[float]`
- Calls `embedding_model.encode(text)`.
- Converts the resulting NumPy-like structure to a plain Python list using `.tolist()`.
- That plain list is what Pinecone expects for storage and querying.

`save_to_pinecone(job, generated_query, embedding) -> None`
- Calls `index.upsert(...)`.
- Stores one vector with:
  - `id`
  - `values` as the embedding
  - `metadata` containing title, company, location, skills, and the generated query
- Uses the configured namespace.

Why metadata matters:
- Pinecone stores the vector for similarity search, but metadata lets the service return meaningful job details later.
- Without metadata, the vector search could identify matching IDs but the frontend would not know what to display.

Search Helper: search_jobs_by_query

`search_jobs_by_query(query, top_k=5, max_score=2.5)` is the candidate-side retrieval helper.

Step by step:
1. Convert the input query into an embedding.
2. Call `index.query(...)` against Pinecone.
3. Request top `k` matches and include metadata.
4. Loop through the matches.
5. Keep only matches whose `score` is less than `max_score`.
6. Return simplified dictionaries with `id`, `score`, and `metadata`.

Why this function exists:
- It centralizes the Pinecone read path.
- Both `/search` and `/search-jobs-from-resume` can reuse it.

Important nuance:
The code filters using `if score < max_score`. In many similarity systems, higher scores are better, but exact semantics depend on the metric configured in Pinecone. The rest of the project also adds a frontend-side threshold of `score > 0.35`. So the effective behavior is to keep results in a middle window rather than simply taking the highest ones.

Interview-ready explanation:
This helper transforms semantic text into a vector, asks Pinecone for the closest stored job vectors, and returns the matches with enough metadata for the frontend to render job cards.

Resume Text Cleaning And Extraction Helpers

The next section of the file handles raw document extraction.

`clean_text(text: str) -> str`
- Returns an empty string for falsy input.
- Replaces the form-feed character `\x0c`.
- Collapses repeated spaces and tabs.
- Collapses excessive blank lines.
- Trims the final result.

This function is small but valuable because resume extraction often produces noisy spacing.

`extract_text_from_pdf(file_like: BytesIO) -> str`
- Resets the stream pointer with `seek(0)`.
- Creates a `PdfReader`.
- Loops page by page.
- Calls `page.extract_text()` on each page.
- Collects non-empty pages.
- Joins them and normalizes them with `clean_text`.

This path works well when the PDF already contains real embedded text.

`extract_text_from_scanned_pdf(file_bytes: bytes) -> str`
- Converts each PDF page to an image using `pdf2image`.
- Runs OCR on each image using `pytesseract.image_to_string`.
- Collects non-empty text.
- Joins and cleans it.

This path exists because many resumes are scans or image-based exports. `PyPDF2` alone cannot read text that is only present visually in pixels.

Skill Extraction And LLM Structuring

`extract_skills_from_text(text: str) -> List[str]`
- Contains a curated list of common skills across programming, cloud, testing, and tools.
- Lowercases the resume text.
- Uses substring checks to find which skills appear.
- Returns a sorted unique list.

Why this helper exists:
- It is a deterministic fallback.
- Even if the LLM fails or misses something, this function can still recover obvious skills.

`get_structured_resume_json(text: str) -> Dict[str, Any]`
- Builds a very explicit prompt asking the model to return only valid JSON.
- Defines a strict schema containing personal info, summary, skills, education, experience, projects, certifications, and extracurricular activities.
- Sends the prompt through `groq_client.chat.completions.create`.
- Uses model name `openai/gpt-oss-120b`.
- Strips accidental code fences using regex.
- Tries to parse the result with `json.loads`.
- If parsing fails, returns an error object with the raw model output.

Why the prompt is so strict:
- LLMs often add prose, markdown fences, or explanatory text.
- The downstream code expects machine-readable JSON.
- The explicit schema makes the output more consistent across resumes.

Merging Skills And Building A Resume Query

`merge_skills(structured_json, fallback_skills) -> List[str]`
- Starts with the fallback keyword-matched skills.
- Looks at the `skills` object returned by the LLM.
- Walks through lists like `technical_skills`, `tools`, and `frameworks`.
- Adds non-empty string items into a set.
- Returns a sorted case-insensitive list.

Why merge instead of choosing one source:
- Deterministic extraction is reliable but limited.
- LLM extraction is richer but can miss or hallucinate details.
- Combining both makes the final skill list more robust.

`build_query_from_resume(structured_json, all_skills) -> str`
- Starts with an empty list of text parts.
- Adds the summary if present.
- Adds up to three experience job titles.
- Adds up to three project titles.
- Pulls technical skills, frameworks, and tools from the structured JSON.
- Appends any remaining merged skills not already included.
- Limits the skill list to 20 items.
- Builds one final string.

Why this function is central:
- Pinecone search works best when the candidate is represented by one semantically rich sentence or paragraph-like query.
- This helper converts raw resume structure into that semantic representation.

Conceptually:
This is the point where the service transforms resume content from a document-parsing problem into an information-retrieval problem.

Endpoint: GET /

The root route is:
- `@app.get("/")`
- function name `health_check`

It simply returns:
- `{"message": "API is running"}`

Why this exists:
- It is a lightweight health endpoint.
- Developers can quickly confirm the service is up without testing the full AI stack.
- It is useful for manual browser checks, load balancers, or simple uptime probes.

Endpoint: POST /generate-query

This endpoint indexes a single job into Pinecone.

Input:
- A JSON request body matching the `JobRequest` model.

Processing steps:
1. Build a readable job sentence with `build_query`.
2. Convert that text to an embedding with `generate_embedding`.
3. Validate that the embedding length is exactly 384.
4. Store the vector and metadata in Pinecone using `save_to_pinecone`.
5. Return job info plus `generatedQuery`, `embeddingLength`, and a status message.

Why the 384 check is there:
- The chosen sentence-transformer model outputs 384-dimensional vectors.
- Pinecone indexes require consistent dimensionality.
- This check catches model or index mismatches early.

Error behavior:
- Any exception becomes an HTTP 500 through `HTTPException`.

How it fits the system:
- Spring Boot calls this route after saving a job in MySQL.
- This service makes that job searchable by semantic meaning, not just exact keyword matching.

Endpoint: GET /search

This endpoint searches indexed jobs using a plain query string.

Inputs:
- `query`: required text
- `top_k`: optional integer, default `3`
- `max_score`: optional float, default `2.5`

Processing:
1. Calls `search_jobs_by_query`.
2. Returns the original query and the filtered matches.

Why this endpoint matters:
- It decouples search from resume parsing.
- Any client can use it, including a frontend, a backend, or a test tool.
- It makes the retrieval layer reusable.

In the current project:
- `SearchJobs.jsx` eventually calls this route after first generating a candidate query from a resume.

Endpoint: POST /analyze-resume

This is the core resume parsing endpoint.

Input:
- one uploaded file under the form field name `file`

Validation:
- if there is no filename, return 400
- if the filename does not end in `.pdf`, return 400

Main flow:
1. Read all file bytes asynchronously.
2. Wrap the bytes in `BytesIO`.
3. Try text extraction using `extract_text_from_pdf`.
4. If too little text comes back, switch to OCR with `extract_text_from_scanned_pdf`.
5. If there is still no usable text, return 400.
6. Save the extracted text into `extracted_texts/<resume>_extracted.txt`.
7. Ask Groq for structured JSON using `get_structured_resume_json`.
8. Extract fallback skills from raw text.
9. Merge the LLM and keyword-derived skills.
10. If the LLM result was valid, inject `all_extracted_skills` into it.
11. Build a semantic `resume_query`.
12. Save the structured JSON into `extracted_texts/<resume>_structured.json`.
13. Return a large JSON payload including filenames, saved paths, structured JSON, all skills, the semantic query, and a preview of extracted text.

Why this endpoint is useful:
- It gives both humans and downstream systems a rich understanding of the resume.
- It produces persistent artifacts that can be inspected later.
- It is the foundation for the recommendation flow.

Endpoint: POST /search-jobs-from-resume

This endpoint combines parsing and search into one call.

It starts almost exactly like `/analyze-resume`:
- validate the upload
- read the bytes
- extract text
- OCR if needed
- request structured JSON from Groq
- merge skills
- build the semantic resume query

Then it goes one step further:
- it calls `search_jobs_by_query(resume_query, top_k, max_score)`

The response includes:
- `filename`
- `resume_query`
- `top_k`
- `max_score`
- `all_extracted_skills`
- `structured_json`
- `matches`

Why this endpoint exists:
- It is convenient for clients that want one API call instead of two.
- It keeps the parsing and search orchestration inside the backend service.

Interesting implementation detail:
- Unlike `/analyze-resume`, this route does not save extracted text and JSON files to disk.
- So the two routes are similar but not identical in side effects.

Generated Files And Observed Outputs

The folder `resume/extracted_texts` currently contains files like:
- `Avinash_Mishra_Resume2.0_extracted.txt`
- `Avinash_Mishra_Resume2.0_structured.json`
- `Yash_Chafle_Resume_extracted.txt`
- `Yash_Chafle_Resume_structured.json`

These files show how the service behaves in practice:
- raw extracted text is preserved
- structured JSON is persisted
- naming is based on the uploaded file name

Why this is useful:
- It makes debugging OCR and LLM parsing easier.
- It creates a transparent audit trail of what the service understood from a resume.
- It can help compare improvements when prompts or extraction logic change.

How The Libraries Work Together

The service is easiest to understand as a pipeline of specialized tools.

Web layer:
- `FastAPI` defines the HTTP routes.
- `UploadFile` and `File(...)` receive multipart uploads efficiently.
- `JSONResponse` sends structured responses.
- `CORSMiddleware` enables frontend browser access.

Configuration layer:
- `python-dotenv` loads secrets from `.env`.
- `os.getenv` reads those secrets.

Document ingestion layer:
- `BytesIO` adapts raw bytes into a file-like stream.
- `PyPDF2` extracts embedded text.
- `pdf2image` converts scanned pages into images.
- `pytesseract` performs OCR on those images.

Reasoning and normalization layer:
- `re` cleans whitespace and strips accidental code fences.
- `json` parses model output.
- `Groq` calls an LLM to convert noisy resume text into structured JSON.

Semantic retrieval layer:
- `SentenceTransformer` converts text to embeddings.
- `torch` and `transformers` sit under the hood to run the model.
- `Pinecone` stores embeddings and returns similar jobs.

In plain words:
FastAPI handles the request, PyPDF2 and OCR read the document, Groq understands it, SentenceTransformer turns it into numbers, and Pinecone uses those numbers to find similar jobs.

Strengths Of The Current Design

Strong parts of the implementation:
- It has a clear fallback strategy for scanned PDFs.
- It separates job indexing from candidate searching.
- It stores metadata along with vectors, which makes result rendering simple.
- It persists extracted artifacts, which helps debugging.
- It validates missing secrets at startup instead of failing later.
- It keeps semantic search independent from the main transactional database.

From an interview perspective, these are good design decisions to highlight because they show awareness of real-world document variability and search quality concerns.

Limitations, Risks, And Improvement Ideas

Current limitations:
- `main.py` is a single large file, so routing, orchestration, prompting, OCR, vector logic, and config are tightly coupled.
- Startup creates external clients and loads the embedding model immediately, which can make cold start slow.
- `.env` secrets are required locally, but there is no secret-management abstraction.
- OCR depends on a hard-coded Windows path for Tesseract.
- LLM output may still be malformed even though the prompt asks for strict JSON.
- There is no retry or timeout strategy shown around Groq or Pinecone calls.
- There are no tests in the `resume` service itself.
- The score filtering logic is not fully intuitive and should be revisited.

Refactors that would improve the service:
- split code into modules such as `config.py`, `schemas.py`, `extractors.py`, `llm.py`, `embeddings.py`, and `routes.py`
- add structured logging
- add request timeouts and graceful error mapping for external APIs
- add unit tests for text cleaning, skill extraction, merge logic, and query building
- make OCR path configurable through environment variables
- add a requirements or lock file for reproducible installs

Interview Explanation Script

If someone asks you to explain this Python service in an interview, you can say:

1. This service is the AI side of the project.
2. It accepts resumes as PDFs and extracts text using PyPDF2, with OCR fallback through pdf2image and Tesseract when the PDF is scanned.
3. It then sends the extracted text to a Groq-hosted LLM to normalize the resume into a consistent JSON structure.
4. From that structure, it builds a semantic query representing the candidate.
5. Jobs are also converted into semantic queries and embedded with `all-MiniLM-L6-v2`.
6. Those embeddings are stored in Pinecone with job metadata.
7. Candidate queries are embedded and compared against stored job vectors to return the most relevant matches.

If you want to sound especially strong, add this:
The Python service is not the system of record. MySQL in the Java backend stores transactional data, while Pinecone in the Python layer stores semantic representations for retrieval.